Programming Assignment 2 (PA2)
Functions, Header Files, File I/O, Arrays, Random Numbers, Recursion

Out: Sunday, February 15th, 2026 -- DUE: Wednesday, March 18th, 2026 at 11:59pm
EC327 Introduction to Software Engineering – Spring 2026

Total: 100 points

- *You may use any development environment you wish, as long as it is ANSI C++ compatible. Please make sure your code compiles and runs properly under the Linux/Unix environment on the PHO 305/307 (or eng-grid) machines before submitting.*
- *Labs may be submitted up to a week late at the cost of a **30% fixed penalty** (e.g., submitting a day late and a week late is equivalent). It is in your best interest to complete as many lab questions as possible before the deadline. If you have missing questions in your original submission, you may complete and submit the missing solutions during the following week. Any submissions after the deadline will be subject to the 30% penalty. No credit will be given to solutions submitted after the 1-week late submission period following the deadline.*
- *Follow the assignment submission guidelines in this document or you will lose points.*

Submission Format (Please Read)

- *Use the exact file names specified in each problem for your solutions.*
- *For revisions and late assignments please append "Late" to the name of the file.*
- *Please do **NOT** submit *.exe and *.o or any other files that are not required by the problem.*
- ***Comment your code (good practice, and required for partial credit!)***
- ***GitHub Instructions TBD on Piazza and/or Blackboard***

Coding Style

As you become an experienced programmer, you will start to realize the importance of good programming style. There are many coding "guidelines" out there and it is important that you become to adopt your own. Naming conventions will help you recognize variableNames, functions, CONSTANTS, Classes etc. Similarly, there are many ways to elegantly format your code so that it is easy to read. All of these issues do not affect compilation and hence are easy to overlook at this stage in your development as a programmer. However, your ability (or inability) to create clean, readable code could be the difference in your career when you leave college.

Here are links you should explore for more on good coding conventions:

http://geosoft.no/development/cppstyle.html#General Recomendations

https://google.github.io/styleguide/cppguide.html

https://isocpp.org/wiki/faq/coding-standards

Q1. Functions, Header Files, File I/O (40 pts)

READ THE WHOLE QUESTION BEFORE YOU START. IT IS LONG AND HAS MANY PARTS.

You need to write a C++ program that will interactively accept commands from the user. Each command is a **single character** followed by one, two, or three parameters and specifies a function that needs to be performed. Your program must do the following:

- Ask the user for the next command code – "`Please enter command code:`"
- Read the command code
- If the command is illegal, print an appropriate error message – "`Invalid command code`"
- else if the command is quit, terminate the program
- else ask the user for the parameters – "`Please enter command parameters:`"
- Read the arguments
- Execute command

Your program must continue this loop until the "quit" command is given.

Commands that you need to include:

F or f	Compute the **F**actorial value	Given an integer number
B or b	Compute the fi**B**onacci number	Given an integer index
R or r	Compute square **R**oots	The sqrt <cmath> function
U or u	Compute the area of Sq**U**are values	Given the length of a side
C or c	Compute the area of **C**ircle values	Given the radius of a circle
E or e	Compute **E**ven numbers	List even numbers between "first" and "last" number provided (ignore delta)
K or k	Compute Luc**K**y numbers	Use random number function
S or s	Compute **S**in values	The sin <cmath> function
N or n	Compute Cosi**N**e values	The cos <cmath> function
X or x	Compute E**X**ponentials	The exp <cmath> function
L or l	Compute Natural **L**ogs	The log <cmath> function
Y or y	Compute N**Y**anCat value	(See function description)
D or d	Compute o**D**d numbers	List odd numbers between "first" and "last" number provided (ignore delta)
I or i	Read from **I**nput file	(See function description)
O or o	Write to **O**utput file	(See function description)
Q or q	**Q**uit program	

You need to define a global program constant named "ENTRIES" which specifies the maximum calculations performed by your program. Initialize this to 10 at the beginning of your program; NOT INSIDE ANY FUNCTION. **(Think about why it cannot be in a function)**

```
extern const int ENTRIES = 10
```

This sets up a variable that is available to be used across multiple files. Look up how to do this properly.

The parameters for F/f and B/b are single integers. There are no parameters for q/Q. The parameters for O/o and I/i are names of the files you want to write to or read from. The parameters for D/d and E/e are the first and last values as integers.

The parameters for the other commands are the double values "first," "last," and "delta." These values will be inputted by the user as three doubles. When a function is called, the corresponding function must be called for all arguments values from first to last, in steps of delta. For instance, for command 'R':

```
Square root of (first)
Square root of (first + delta)
Square root of (first + delta*2)
Square root of (first + delta*3)
…
Etc
```

If delta is ≤ 0 or first > last, then no computation takes place. **Print "No computation needed."** If these special cases are not triggered, then compute the values until you have completed the full range of values or you have computed more than **ENTRIES**, whichever comes first. Your program must check for legal command codes, legal delta values, and make sure first ≤ last. However, you DO NOT need to check that a given argument range is legal for the function being called. For instance, you do not need to check that sqrt is being called with positive argument. You may assume that the user will only enter one keystroke when asked for the command. You should always include the first value in your computation, then proceed in delta increments until you hit the last value or exceed the ENTRIES value. **If a delta increment exceeds the "last" value, just use the last value for your final computation.**

All the functions must be declared in a *.h file and defined in a *.cpp file. They will be defined as follows:

```
pa2Functions.h
pa2Functions.cpp
```

pa2Functions.h will hold all the function prototypes while **pa2Functions.cpp** will hold the implementations. In addition to these two files, you will have a **Q1.cpp** where you will run the main function and call the functions in your library.

pa2Functions.h must include the following prototypes, **EXACTLY** as given here.

```
void initialize();
```

This function must print the program output header. The output header should contain the name and number of the course, the semester and year, the assignment name, and the value of

ENTRIES. The values of ENTRIES must be obtained from Q1.cpp. This function is called from main().

(Header Sample)

```
EC327: Introduction to Software Engineering
Fall 2026
Programming Assignment 2
Value of Entries is: 10
```

`bool checkCode(char);`

This function should return true if the command code in parameter "code" is a legal code character; otherwise, the function should return false. You have to use this function to check your user input. This function is called from main().

`void writeDataToFile(const char *);`

This function will write the output of the program to a file specified by user input. This only works for input provided AFTER this command has been provided. The output being written to a file should also be displayed to the console as well. Writing to a file ends when the quit command is provided.

`void readDataFromFile(const char *);`

The function will read commands from a file specified by user input. This function is called from main(). The commands in this file should be formatted each on two lines consistent with how the user would enter them from the command line.

e.g.

```
f
10
R
10 20 .2
```

`int factorial(int);`

This function returns the factorial value of the given integer argument.

`int fibonacci(int);`

This function returns the Fibonacci number at the index given by the provided argument.

`double findSqrtValue(double);`

This function returns the square root of the argument.

`double naturalLog(double);`

This function returns the natural log of the provided argument.

`double areaCircle(double);`

This function returns the area of a circle provided the radius.

`double areaSquare(double);`

This function returns area of a square provided the length of its side.

`int findNextOddValue(int);`

This function returns an odd number **closest and higher** to the provided number.

`int findNextEvenValue(int);`

This function returns an even number **closest and higher** to the provided number.

`double findNyanCatValue(double);`

If this function takes in myNum this function with return

$(4 * myNum)^{myNum} + myNum + 10$

`double doMath(double, char);`

This function performs either sin, cosine, or the exponential function for the value provided. The operation to perform is based on the char that is provided.

`double lucky(double);`

This function returns a random number using the **system time and the argument** as a seed.

Final requirements

- All functions (other than main) must be prototyped (declared) in **pa2Functions.h**
- All functions (other than main) must be implemented (defined) in **pa2Functions.cpp**
- Main function must be implemented in **Q1.cpp**
- Comment your code!!!
- All input and output (other than file I/O) must be done with `cin` and `cout`, respectively

<u>Submission requirements:</u> You should submit **three files:** `Q1.cpp`, `pa2Functions.cpp` and `pa2Functions.h`. **More information regarding GitHub is provided separately.**

Q2. File I/O, Arrays, Random Numbers, and Functions (40 pts)

1. Write a function named `WriteRandomData()` that generates N random integers from 0 to M (inclusive) and writes them to a text file named `data.txt`, one number per line. You should use the following function prototype:

```
void WriteRandomData(int N, int M, const char *filename);
```

2. Write a second function called `ReadData()` that reads the data from the specified text file, and also finds the size (number of data elements). You can use the following function prototype:

```
void ReadData(const char *filename, int &size, int myArray[]);
```

3. Write a function called `DoubleAndReverse()` that doubles the size of an integer array and fills the second half with a copy of the first half in reverse order. You can use the following function prototype:

```
int * DoubleAndReverse (int *list, int size);
```

Example: Given array: [1 2 3 4 5]
The resulting array obtained by `DoubleAndReverse` function: [1 2 3 4 5 5 4 3 2 1]

The function should return a pointer to an integer array that has a size equal to 2*size. The values in the new array at indices from 0 to size-1 should have the same values as in the original input array in the same order. The rest of the values in the new array should be the reverse order of the original array.

4. Write functions `getMedian()` and `getLargest()` to compute the median and largest number in an array. These functions should accept an array and size as an argument and return a double and an integer respectively. The function prototypes are left to you to figure out.

5. Write a `main()` function that creates the `data.txt` file, assumes the `data.txt` file is given in the same folder as the code files, reads that file, calculates the size of an array, calls the `DoubleAndReverse` function, prints that array, and calculate the median and largest values. For example, your final output screen should look like this (for some random `data.txt` file):

```
Writing file: data.txt
Reading file: data.txt
Array size is: 5
Doubled and reversed is [1 2 3 4 5 5 4 3 2 1]
Median is: 3.5
Largest is: 5
```

We will test your code on our `numbers.txt` file. Assume the file can be **any** size between (and including) 0 and 1000.

<u>Submission requirements:</u> You should submit **five separate files:** `Q2.cpp`, `FileFunctions.h`, `FileFunctions.cpp`, `Statistics.h`, `Statistics.cpp`.

- `Main()` and `DoubleAndReverse()`, should be saved in `Q2.cpp`.

- `WriteRandomData()` and `ReadData()` should be stored in a single `FileFunctions.cpp` file with the header file named `FileFunctions.h`.
- `getMedian()` and `getLargest()` should be stored in a single `Statistics.cpp` file with the header file named `Statistics.h`.

Important: As in Q1, you can use `<cmath>` library in your main program but you **must not** use `<cmath>` library (or any other math library) in `Statistics.cpp` file.

Q3. Array of pointers and dynamic memory allocation (30 points)

Write a function called **ReverseMultiply** that doubles the size of an integer array and fills the second half with a copy of the first half in reverse order and adds a final additional location for the product of all the numbers.

Example: Given array: [1 2 3 4 5]
The resulting array obtained by ReverseMultiply function: [1 2 3 4 5 5 4 3 2 1 14400]

The function prototype is as follows: **int * ReverseMultiply (int *list, int size);**

The function should return a pointer to an integer array that has a size equal to 2*size + 1. The values in the new array at indices from 0 to size-1 should have the same values as in the original input array in the same order. The rest of the values in the new array should be the reverse order of the original array. The final value should be the product.

Write a function called **ReverseAdd** that creates an array with the original values and then adds new entries that are the sum of sets of numbers in the array starting from the original "end" moving forward.

Example: Given array: [1 2 3 4 5]
The resulting array obtained by ReverseAdd function: [1 2 3 4 5 9 7 5 3]

Example: Given array: [10 2 7 4 9]
The resulting array obtained by ReverseAdd function: [10 2 7 4 9 13 11 9 12]

The function prototype is as follows: **int * ReverseAdd (int *list, int size);**

The function should return a pointer to an integer array that has a size equal to 2*size-1. The values in the new array at indices from 0 to size-1 should have the same values as in the original input array in the same order. The rest of the values in the new array should be the sum of sets of numbers starting from the end of the original array.

Your main function (in Q3.cpp along with the ReverseMultiply and ReverseAdd function) should request the size of the original array and then allow the user to input the values. It should then print out the original array and address along with the new arrays and addresses.

For example:

```
Enter the number of entries: 3 <newline>
Entry 0 is:  3 <newline>
Entry 1 is:  7 <newline>
Entry 2 is:  20 <newline>
*****
Original array is: 3 7 20 and the address of the zero element is: <something in hex>
One array is: 3 7 20 20 7 3 176400 and the address of zero element is: <something in hex>
Two array is: 3 7 20 27 10 and the address of zero element is: <something in hex>
```

<u>Submission requirements:</u> You should submit **one file:** `Q3.cpp`